Digital Twin-Driven Deep Reinforcement Learning for Energy-Aware CNC Milling Parameter Optimization

ME6324 – AI in Manufacturing

Course Project Report

Abhinand
Department of Mechanical Engineering
Indian Institute of Technology Madras
abhinand@iitm.ac.in

October 21, 2026

Git Revision: `d76603b` (main branch)

This report is generated from the project repository at `G:/My Drive/IITM/AI in Mfg`. All results are reproducible from the committed artifacts using the provided scripts.

Contents

1 Introduction **3**
- 1.1 Problem Statement . . . 3
- 1.2 Proposed Approach . . . 3
- 1.3 Contributions . . . 4

2 Related Work **4**

3 Digital Twin Physics **5**
- 3.1 Geometry and Kinematics . . . 6
- 3.2 Tool Wear (Extended Taylor Model) . . . 6
- 3.3 Surface Roughness (Empirical Regression) . . . 6
- 3.4 Energy Consumption . . . 6
- 3.5 Stochastic Tool Breakage . . . 7

4 Environment Design **7**
- 4.1 Registrations . . . 7
- 4.2 State Space . . . 8
- 4.3 Action Space . . . 8
- 4.4 Reward Function . . . 9
- 4.5 Episode Logic . . . 9
- 4.6 Domain Randomization . . . 9

5 Reinforcement Learning **10**
- 5.1 Algorithm . . . 10
- 5.2 Training Runs . . . 10
- 5.3 Monte Carlo Evaluation . . . 11

6 Classical Baselines **12**

7 Results **12**
- 7.1 Production Policy – Monte Carlo Sweep (100 Scenarios) . . . 13
- 7.2 Energy Weight Ablation – Pareto Frontier . . . 16
- 7.3 Runtime Weight Sweep of the Production Policy . . . 17
- 7.4 One Policy against the Per-Weight Policies . . . 18
- 7.5 Baseline Comparison . . . 18
- 7.6 Robustness Against the Classical Controller . . . 19
- 7.7 Batch Against the Part . . . 20
- 7.8 Controller Initialization Experiment . . . 21
- 7.9 Transfer Experiment . . . 23

8 Interactive Digital Twin Workstation **23**

9 Software Engineering Practices **24**
- 9.1 Architecture . . . 24
- 9.2 Behavior Cloning as an Initialization . . . 24
- 9.3 Reproducibility . . . 24
- 9.4 Testing . . . 25
- 9.5 Code Quality . . . 25

10 Discussion **25**
10.1 What Worked 25
10.2 Limitations 26
10.3 Future Work 27

11 Conclusion **27**

A Reproducibility Commands **29**

B Project Structure **30**

C Hyperparameter Sensitivity **32**

D Energy Model Derivation **32**

1 Introduction

1.1 Problem Statement

CNC face milling is a fundamental manufacturing operation where a rotating cutter removes material from a workpiece in multiple passes. The cutting parameters – spindle speed (n), feed per revolution (f_r), and axial depth of cut (a_p) – must be chosen to balance four competing objectives:

1. **Productivity:** Maximize material removal rate (MRR $= v_c \cdot a_p \cdot f_r$)
2. **Energy Efficiency:** Minimize total electrical energy consumption (E_{total}), accounting for both machine idle power and cutting power.
3. **Surface Quality Assurance:** Strictly adhere to finished workpiece roughness requirements ($R_a \leq 1.6$ μm).
4. **Tool Wear and Breakage Avoidance:** Limit cumulative flank wear ($V_B \leq 0.3$ mm) and mitigate catastrophic tool failure risks.

Traditional approaches use fixed parameters from handbooks or static optimization (e.g., NSGA-II) that pick one parameter set for the entire batch. These cannot adapt to the *state* of the process – tool wear accumulates, workpiece geometry changes, and material properties vary between batches.

1.2 Proposed Approach

This project develops a **Digital Twin** of the milling process – a high-fidelity physics-based simulator grounded in empirical manufacturing literature – and trains a **Deep Reinforcement Learning (DRL)** agent to dynamically optimize cutting parameters pass-by-pass. The agent observes the current state (flank wear, remaining stock, previous parameters) together with the context it is optimizing for (the energy weight and the material), and selects the next pass parameters to maximize a reward that balances throughput and energy while respecting constraints.

Key innovations:

- **Speed-dependent specific cutting energy** $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$ creates a genuine trade-off: faster cutting increases energy per unit volume but reduces idle energy.
- **Domain randomization** during training: material/tool coefficients, workpiece geometry, and initial wear are sampled per episode from literature-calibrated ranges.
- **One environment, one policy:** The energy weight w_2 and the material context live in the observation, so a single PPO policy covers the whole weight range and a whole distribution of materials.
- **A fair classical baseline:** A part-level controller that maximizes the expected episode reward over the same physics, with the breakage risk, the step penalty and the completion bonus priced.
- **Robustness experiment:** The same two arms on the same scenarios in three regimes: the controller with every advantage, the controller with a wrong model of the material, and the controller with a noisy wear measurement.
- **Batch experiment:** A second environment in which one tool cuts a batch of parts, the tool life is shared across the parts and the tool change is priced.

- **Transfer experiment:** Measures how much a policy trained on one material knows about a harder, faster-wearing material.

1.3 Contributions

1. A Gymnasium-compliant milling environment with explicit physics, energy-aware reward, and domain randomization.
2. A batch environment in which one tool cuts a batch of parts, the tool life is shared across the parts and the tool change is priced.
3. A trained PPO policy evaluated via 100-scenario Monte Carlo sweep with 95% confidence intervals.
4. Energy-weight ablation study (7 weights $\times$ 3 seeds) mapping the empirical Pareto frontier.
5. Comparison against 5 classical baselines, all on the same episodes:
   Constant-High, Constant-Safe, Rule-Based, NSGA-II Static, Receding-Horizon.
6. A robustness experiment against the receding-horizon controller in three regimes: the controller with every advantage, the controller with a wrong model of the material, and the controller with a noisy wear measurement.
7. A batch experiment: the batch context is worth 8% fewer tool changes and 3.5% less energy at the same completion, and the per-part controller needs 207.5 s of machine time for a batch of ten parts against the batch-aware policy's 113.8 s.
8. Transfer experiment on a harder alloy: a policy trained across a distribution of materials is the best arm there without any adaptation.
9. A controller initialization experiment whose result reversed: the initialization is now harmful rather than harmless (Section 7.8).
10. Interactive Streamlit workstation with 2D stock depletion, micro-topography, G-code generation, and multi-controller comparison.
11. Full reproducibility: all results regenerated from committed artifacts by `experiments/summarize_results.py`.

2 Related Work

This project positions itself against the papers held in `docs/papers/`:

Table 1: Position against the reference papers held in `docs/papers/`

Capability	Mo et al. (2025) *J. Manuf. Proc. 156*	Wang et al. (2025a) *J. Manuf. Sys. 78*	**This Work**
Dynamic parameter optimization	✓	✓	✓
Tool wear in the state	✓	✗	✓
Surface roughness constraint	✓	✗	✓
Energy as an objective	✗	✗	✓
Speed-dependent cutting energy	✗	✗	✓
Domain randomization	✗	✗	✓
Conditioned policy (single → frontier)	✗	✗	✓
Transfer experiment across materials	✗	✗	✓
Receding-horizon baseline	✗	✗	✓
Robustness against a model-based controller	✗	✗	✓
Monte Carlo robustness (100 scenarios)	✗	✗	✓
Interactive digital twin workstation	✗	✗	✓

Both reference papers treat energy as related work rather than as an optimization target: in each of them energy appears in the discussion of prior art, not among the objectives. Mo et al. optimize the surface roughness and the production time of a multi-pass turning operation and carry the tool condition and the tool damage in the decision variables, so the wear and the roughness marks carry over. Wang et al. (2025a) build their state from sequences of machining parameters and cutting force features, and their objectives from the dimensional error of the machining features and the machining time; neither the tool wear nor the roughness enters their formulation, so only the dynamic parameter optimization carries over. The third paper in `docs/papers/` (Wang et al., 2025b) solves the discrete step sequencing problem, so continuous cutting parameters are outside its scope. None of the three carries a Taylor wear model, so the wear coefficient of this work is anchored on the tool life that Wang et al. report rather than on a wear equation taken from a paper (Section 3.2).

The speed-dependent cutting energy model (Section 3) is the key enabler here: without it the energy of a part is fixed by the volume removed, both objectives point the same way, and no weight on the energy term can open a trade-off. Putting the energy weight and the material context in the observation reduces the ablation cost from 21 training runs to one. The receding-horizon controller provides a fair classical benchmark that uses the same physics, re-optimizes at every pass and maximizes the same expected episode reward, and the robustness experiment of Section 7.6 puts the two on the same scenarios in the three regimes a plant actually runs in.

3 Digital Twin Physics

All equations are implemented in `MillingDT/config/physics.py` and used by the environment, baselines, simulator, and evaluation scripts – a single source of truth. The state the agent sees is the 11-dimensional observation of Section 4.2: eight physics features followed by the conditioning context, and the context is present in every task, so the equations below are the same equations for every task.

3.1 Geometry and Kinematics

Cutter diameter $D = 50$ mm, cutting length $L_{\text{cut}} = 100$ mm, total machining allowance $\kappa_{\text{total}} = 5.0$ mm.

$$v_c = \frac{\pi D n}{1000} \qquad [\text{m/min}] \tag{1}$$

$$\text{MRR} = v_c \cdot a_p \cdot f_r \qquad [\text{mm}^3/\text{min}] \tag{2}$$

$$t_{\text{cut}} = \frac{L_{\text{cut}}}{n f_r} \qquad [\text{min}] \tag{3}$$

$$N_{\text{passes}} = \left\lceil \frac{\kappa_{\text{total}}}{a_p} \right\rceil \qquad [\text{integer}] \tag{4}$$

3.2 Tool Wear (Extended Taylor Model)

Flank wear rate per pass:

$$\Delta V_B = C_{\text{wear}} \cdot v_c^{\alpha} \cdot f_r^{\beta} \cdot a_p^{\gamma_{\text{wear}}} \cdot t_{\text{cut}} \tag{5}$$

with calibrated coefficients $C_{\text{wear}} = 1.0 \times 10^{-2}$, $\alpha = 0.3$, $\beta = 0.2$, $\gamma_{\text{wear}} = 0.15$. Cumulative wear $V_{B,k} = \sum_{i=1}^{k} \Delta V_{B,i}$.

The coefficient is anchored on a tool life, not on a wear equation. None of the three reference papers carries a Taylor wear model, so there is no published wear equation to calibrate the coefficient against. What Wang et al. (2025a) do report is a tool life: 40 parts cut with 5 identical tools, which is about 8 parts per tool. The coefficient is set so that the tool life at the reference cutting condition is about 9 parts per tool, which puts the tool life inside the range where a batch can run out of a tool. The previous value $C_{\text{wear}} = 1.1 \times 10^{-4}$ gave a tool life of about 870 parts, so the wear constraint did nothing inside a part and the tool life was irrelevant to a batch; the wear model was inert.

The exponents are the exponents of the wear rate, not of the Taylor tool life. The classical tool life equation carries the exponents $(4.0, 0.8, 3.2)$, and with them the tool life spans six orders of magnitude over the machine envelope, which is not a usable envelope either. The exponents used here describe how fast the wear accumulates, so the tool life varies only by a factor of a few across the envelope: a faster cut barely shortens the tool life in this model. That is a known weakness of the wear model and it is carried into the limitations of Section 10.2. It is the reason the wear constraint acts mainly on the aggregate of a batch rather than on the choice of a single pass.

3.3 Surface Roughness (Empirical Regression)

$$R_a = k_1 f_r^2 + k_2 V_B + \frac{k_3}{v_c} \tag{6}$$

with $k_1 = 12.5$, $k_2 = 0.8$, $k_3 = 0.5$. The finished surface (last pass) must satisfy $R_a \le R_{a,\max} = 1.6$ μm.

3.4 Energy Consumption

Specific cutting energy grows with cutting speed:

$$k_c(v_c) = k_{c0} \left(\frac{v_c}{v_{\text{ref}}} \right)^{m}, \quad P_{\text{cut}} = k_c(v_c) \cdot \text{MRR}, \quad P_{\text{total}} = P_{\text{idle}} + P_{\text{cut}} \tag{7}$$

with $P_{\text{idle}} = 500$ W, $k_{c0} = 2.5$ W $\cdot$ min/mm^3, $m = 0.6$, v_{ref} from the reference cutting condition. Energy per pass: $E_{\text{pass}} = P_{\text{total}} \cdot t_{\text{cut}}$. Total energy per part: $E_{\text{part}} = \sum_{k=1}^{N_{\text{passes}}} E_{\text{pass},k}$.

Proposition 1 (Interior Minimum)**.** *With $m > 0$, the energy per part $E_{part}(v_c)$ has an interior minimum in v_c. The cutting term rises with v_c while the idle term falls, creating a genuine trade-off that the energy weight w_2 can select.*

This is validated by test `test_part_energy_has_an_interior_minimum_in_speed`.

3.5 Stochastic Tool Breakage

Probability of catastrophic breakage per pass, applied only once the flank wear passes a threshold:

$$p_{\text{break}} = 0.3 \cdot \underbrace{\frac{V_B - 0.15}{V_{B,\max} - 0.15}}_{\text{wear factor}} \cdot \underbrace{\left(\frac{\text{MRR}}{\text{MRR}_{\max}}\right)^2}_{\text{load factor}} \qquad (V_B > 0.15 \text{ mm, else } 0) \tag{8}$$

where $\text{MRR}_{\max}$ is the removal rate of the machine envelope at its upper bounds. The draw comes from the environment's own generator, which `reset(seed=...)` seeds, so a scenario replayed with the same seed breaks in the same place. If triggered, the episode terminates with a -100 reward.

4 Environment Design

4.1 Registrations

Two Gymnasium environments are registered in `MillingDT/env/__init__.py`. The per-part one is exported as the constant `MillingDT.env.ENV_ID` and the batch one as `MillingDT.env.BATCH_ENV_ID`:

1. `MillingDTEnv-v1`: the part environment. The observation is always $\mathbb{R}^{11}$: the eight physics features followed by the conditioning context (the energy weight w_2, the wear coefficient C_{wear} and the roughness coefficient k_1). The flags `randomize` and `randomize_weight` decide whether the scenario and the energy weight are drawn per `reset()`. A task with a pinned weight is the same environment with a constant in the context, so there is a single observation shape across the whole project.

2. `MillingDTBatchEnv-v1`: the batch environment. One episode is a batch of ten parts cut with one tool. The observation is $\mathbb{R}^{13}$: the eleven features of the part the tool is cutting, then the fraction of the tool life that is left and the fraction of the batch that is still to cut. The tool is replaced when it breaks or reaches its wear limit, the wear of the new tool starts at zero, and the replacement costs 500 reward units against the 200 a finished part is worth. The physics is not duplicated: the batch environment wraps the part environment and adds the tool accounting.

Two optional arguments make the environment behave like a plant rather than like a simulator with perfect knowledge. The argument `belief` sets what the setup *believes* the material is, while the physics keep running on the truth, and the argument `obs_noise` puts noise on the two measured state features, the flank wear and the remaining stock. Both are used by the robustness experiment of Section 7.6.

4.2 State Space

Table 2: Observation (11 dimensions), every feature mapped onto $[0, 1]$

Index	Feature	Normalization
0	Flank wear V_B	tanh squash over $[0, V_{B,\max}]$
1	Remaining stock κ_{rem}	tanh squash over $[0, \kappa_{\text{total}}]$
2	Previous spindle speed n	$(n - n_{\min})/(n_{\max} - n_{\min})$
3	Previous feed f_r	$(f_r - f_{\min})/(f_{\max} - f_{\min})$
4	Previous depth a_p	$(a_p - a_{p,\min})/(a_{p,\max} - a_{p,\min})$
5	Previous MRR	$\text{MRR}/\text{MRR}_{\max}$, clipped
6	Previous roughness R_a	tanh squash over $[0, R_{a,\max}]$
7	Tool broken	1 if broken, else 0
8	Energy weight w_2	$w_2 \in [0, 1]$
9	Wear coefficient C_{wear}	over the scenario band $[0.6, 1.4]$ of nominal
10	Roughness coefficient k_1	over the scenario band $[0.8, 1.2]$ of nominal

Every feature is normalized against the *scenario's own* limits, so the observation is scale free across geometries and materials. The tanh squashing keeps the gradient magnitude when a constraint is violated, instead of clipping the value to 1.

The context occupies the last three slots in every task. A policy trained with a pinned weight sees a constant there, a policy trained with the weight drawn sees the weight it is being asked for, and a policy trained across the material distribution sees the material it is cutting. The shape never changes, so one trained policy can be asked for any weight and any material at runtime. The two material slots carry what the setup *believes* the material is, not what it is: a plant that does not know its material passes a belief that differs from the truth while the physics keep running on the truth. Of the eight state features only two are measured, the flank wear and the remaining stock, and only those two carry the measurement noise.

4.3 Action Space

Continuous 3D box $\mathcal{A} = [-1, 1]^3$, mapped to physical parameters:

$$n = n_{\min} + \frac{a_1 + 1}{2}(n_{\max} - n_{\min}) \tag{9}$$

$$f_r = f_{\min} + \frac{a_2 + 1}{2}(f_{\max} - f_{\min}) \tag{10}$$

$$a_p = a_{p,\min} + \frac{a_3 + 1}{2}(a_{p,\max} - a_{p,\min}) \tag{11}$$

The policy output is unbounded; the environment clips to $[-1, 1]$ before mapping, guaranteeing the machine envelope is never violated.

4.4 Reward Function

The per-step reward is:

$$
\begin{aligned}
r = \; & \underbrace{100\left(w_1 \frac{\mathrm{MRR}}{\mathrm{MRR}_{\max}} - w_2 \frac{E_{\mathrm{pass}}}{E_{\mathrm{ref}}}\right)}_{\text{Objective}} + \underbrace{20\,\Delta\left(\frac{\kappa_{\mathrm{removed}}}{\kappa_{\mathrm{total}}}\right)}_{\text{Progress}} \\
& - \underbrace{50\,\mathbb{1}[V_B > V_{B,\mathrm{safe}}]}_{\text{Wear penalty}} - \underbrace{100\,\mathbb{1}[\text{broken}]}_{\text{Breakage}} \\
& - \underbrace{50\,\mathbb{1}[\text{finishing pass} \wedge R_a > R_{a,\mathrm{safe}}]}_{\text{Roughness penalty}} - 0.05
\end{aligned}
\tag{12}
$$

plus, on a part finished inside spec, a +200 completion bonus and a Pareto margin bonus of at most +5:

$$
\mathrm{HV} = \max\!\left(0, 1 - \tfrac{T_{\mathrm{total}}}{T_{\mathrm{ref}}}\right) \cdot \max\!\left(0, 1 - \tfrac{R_a}{R_{a,\mathrm{safe}}}\right) \cdot 5 \tag{13}
$$

All of these magnitudes live in the `reward` section of `MillingDT/config/physics_constants.yaml`, so they are in one place and the file states that they are a design decision. **The scale of the objective term matters as much as the weights.** The objective term is what decides the operating point, so it has to be worth a sensible fraction of the value of a finished part. With the objective scale at 5 the whole throughput and energy trade-off of a part was worth about 1.2 points of reward against a completion bonus of 200, so a policy that finished every part at any cost scored well and the energy weight had almost nothing to select. At 100 the trade-off is worth about fifty points per part, a quarter of the value of a part, and the completion bonus still dominates the decision to finish. The constraint penalties (50 for a pass above the wear safety limit, 100 for a broken tool, 50 for a finishing pass outside the roughness safety limit) sit above the progress bonus of 20, so that breaking a constraint is never worth a throughput or energy gain.

$R_{a,\mathrm{safe}} = 0.85\,R_{a,\max}$, $w_1 + w_2 = 1$, and E_{ref} is the energy of a pass under the reference condition, recomputed for every scenario from that scenario's diameter, cutting length and energy coefficients. The reward is validated by `test_reward_matches_an_independent_calculation`, which recomputes it from `MillingDT/config/physics.py`, independently of the code that assembles it, and asserts agreement to 10^{-9}.

4.5 Episode Logic

- `terminated=True`: Part finished ($\kappa_{\mathrm{rem}} \leq 0$), tool broken, or wear exceeded.
- `truncated=True`: Maximum steps (200) reached.
- The roughness penalty is charged once, at the finishing pass, and only when that pass ends outside $R_{a,\mathrm{safe}}$. Intermediate passes carry no roughness penalty; the wear penalty is charged on every pass whose cumulative flank wear exceeds $V_{B,\mathrm{safe}}$.

4.6 Domain Randomization

When `randomize=True`, each `reset()` samples a new scenario from `MillingDT/config/scenarios.py`:

- Workpiece: $\kappa_{\mathrm{total}} \in [2, 15]$ mm, $D \in [40, 150]$ mm, $L_{\mathrm{cut}} \in [50, 200]$ mm
- Material/Tool: C_{wear} at $[0.6, 1.4]\times$ nominal, k_1, k_2, k_3 each at $[0.8, 1.2]\times$ nominal
- Initial wear: $V_{B,0} \sim \mathcal{U}[0, 0.8]\,V_{B,\max}$

The same tables are used for Monte Carlo evaluation, so the policy is measured on the distribution it was trained on. Measured on the trained policy, the eight physics features the policy receives on the two sets agree to 0.041 in the mean of their normalized values, against a standard deviation of 1.

5 Reinforcement Learning

5.1 Algorithm

Proximal Policy Optimization (PPO) with MLP policy, implemented via Stable-Baselines3.

Table 3: PPO hyperparameters

Parameter	**Value**
Policy	MlpPolicy (2 hidden layers, 256 units each)
Learning rate	3×10^{-4} (linear schedule to 0)
n_steps	2048
Batch size	64
n_epochs	10
γ	0.99
GAE λ	0.95
Clip range	0.2
Entropy coefficient	0.02
Value function coefficient	0.5
Max grad norm	0.5
VecNormalize	`norm_obs=True, norm_reward=True`
Environments	4 parallel (`SubprocVecEnv`)
Device	CPU (2.5× faster than GPU for this MLP)

Training uses a custom `ProgressQueueCallback` that sends a heartbeat every 1000 steps to the supervisor (`run_experiments.py`); a dropped heartbeat triggers worker termination.

5.2 Training Runs

1. **Production policy:** 1,000,000 steps, seed 42, `--randomize --randomize-weight`. Both the machining scenario and the energy weight are drawn per episode, so one network covers the whole weight range and the whole material distribution. Model at `results/run_PPO_seed42/`. This is the policy the dashboard, the Monte Carlo sweep, the runtime weight sweep and the baseline comparison all use.

2. **Ablation grid:** 7 weights $w_2 \in \{0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}\times$ 3 seeds $\{42, 43, 44\}$, 150,000 steps each, trained in parallel via `experiments/run_experiments.py`. Every ablation policy is trained with its energy weight **pinned** and is evaluated at the weight it was trained for, so it is always asked the question it was trained for. The model directories follow the pattern `results/run_ablation_w2_0_4_seed42`, one per weight and seed.

3. **Batch-aware policy:** 1,000,000 steps, seed 42, on the batch environment (`--env-id MillingDTBatchEnv-v1 --randomize`). The observation carries the tool life left and the fraction of the batch still to cut. Model at `results/run_PPO_batch_seed42/`.

4. **Policy trained with a noisy wear measurement:** 1,000,000 steps, seed 42, `--randomize --randomize-weight --obs-noise 0.05`. The noise is on the

two measured state features, the flank wear and the remaining stock. Model at `results/run_PPO_noisystate_seed42/`.

All training paths use the same script, `experiments/train_rl.py`: `--randomize` draws a new machining scenario per episode, `--randomize-weight` draws a new energy weight per episode, `--w2` pins the weight of the task, `--env-id` selects the environment and `--obs-noise` puts noise on the measured state. The controller initialization experiment trains its own arm through the same script with the flag `--init-from-controller`.

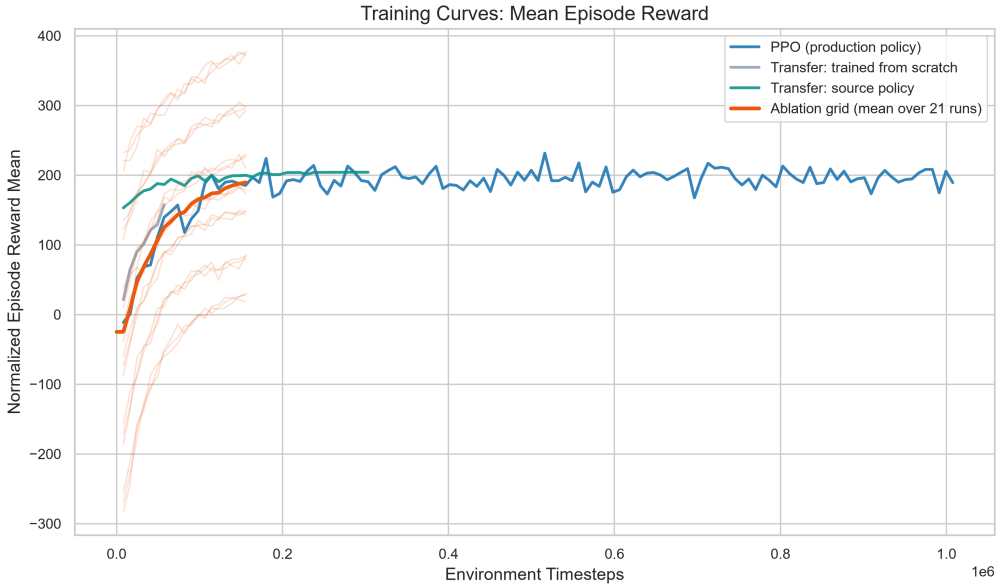


Figure 1: Training curves of every run: the mean episode reward against the environment steps. The production policy in blue, the two arms of the transfer experiment, and the mean over the 21 ablation policies in orange with the individual runs drawn faintly behind it.

Figure 1 shows what the budgets buy. The production policy reaches the level of about 200 inside its first 200,000 steps and then oscillates around it for the remaining 800,000, so the size of the budget buys stability rather than a higher reward. The 21 ablation policies reach the same level within their 150,000 steps, the individual runs drawn faintly behind their mean spread widely, and the two transfer arms converge inside their 300,000 step budget. The vertical axis is the normalized reward mean of `VecNormalize`, so it does not carry the scale of the episode reward reported in the evaluation tables.

5.3 Monte Carlo Evaluation

`python -m experiments.evaluate --scenarios 100` runs the production policy over 100 randomized scenarios (fixed seed 42). Each scenario is a full machining episode. Reports mean ± std and 95% CI (t-distribution) for:

- MRR, Energy/part, Cycle time
- Max R_a, Max V_B
- Constraint violation rates (worst pass vs finished surface)
- Tool breakage rate, Part completion rate

6 Classical Baselines

Five non-RL baselines implemented in `experiments/run_baselines.py`:

1. **Constant-High:** $n = 8000, f_r = 0.35, a_p = 2.5$ (upper bounds).
2. **Constant-Safe:** $n = 2000, f_r = 0.05, a_p = 0.1$ (lower bounds).
3. **Rule-Based Degradation:** Feed at maximum, spindle speed reduced by 20% every second pass, as a shop-floor heuristic for tool degradation.
4. **NSGA-II Static:** Multi-objective optimization (pymoo) picking *one* parameter set for the whole part. Decision variables $[n, f_r, a_p]$. Objectives: maximize MRR, minimize Energy. Constraints: $R_{a,\text{final}} \leq R_{a,\max}$, $V_{B,\text{final}} \leq V_{B,\max}$.
5. **Receding-Horizon Controller:** A vectorized grid search over (n, f_r, a_p) that re-solves at every pass, simulating the remaining passes of the part with the current V_B and κ_{rem}. It is a part-level maximizer of the expected episode reward: for every candidate it prices the objective term, the progress bonus, the wear penalty, the step penalty, the completion bonus, the Pareto margin and the breakage probability, and it keeps the candidate with the best expected value. The plan of a part is two parameter sets, a roughing feed and a finishing feed, because the reward penalizes the roughness of the finishing pass only and a plan that holds one feed for the whole part would have to satisfy the roughness limit on the worst pass. This is the strongest classical baseline.

The controller is a baseline in the strict sense: it optimizes the same quantity the environment pays, over the same physics, and it is given the model and the state it is handed. It is not given anything the policy does not have, and it is not a stand-in for the reward. Its quality is the measure of the comparison: on the same scenarios it went from 184.3 to 248.9 when it was changed from a per-pass maximizer of the objective term into a part-level maximizer of the expected episode reward. All baselines use the *same physics* (`MillingDT/config/physics.py`) and are evaluated on the same 50 episodes of the nominal machining condition, at the same energy weight $w_2 = 0.4$ at which the learned policy is queried.

7 Results

All tables below are regenerated from committed CSVs by
`python -m experiments.summarize_results`.

7.1 Production Policy – Monte Carlo Sweep (100 Scenarios)

Table 4: Production policy Monte Carlo results (100 scenarios)

Metric	**Mean**	**Std**	**95% CI**
Material removal rate (mm^3/min)	1608.49	550.69	[1499.22, 1717.76]
Energy per part (kJ)	66.40	48.03	[56.87, 75.93]
Cycle time per part (s)	11.43	6.93	[10.06, 12.81]
Max surface roughness (μm)	1.610	0.178	[1.575, 1.645]
Max flank wear (mm)	0.1407	0.0672	[0.1274, 0.1540]
Constraint violations			
Roughness violations, worst pass	55.0%		[45.2%, 64.8%]
Roughness violations, finished surface	4.0%		[0.2%, 7.8%]
Roughness > safety limit, worst pass	87.0%		[80.4%, 93.6%]
Roughness > safety limit, finished surface	4.0%		[0.2%, 7.8%]
Flank wear violations ($V_B > 0.3$ mm)	0.0%		[0.0%, 0.0%]
Tool breakage	6.0%		[1.3%, 10.7%]
Parts completed within spec	94.0%		
Scenarios	100		
Mean operating point	7907 RPM, 0.318 mm/r, 2.07 mm, 4.0 passes		

Key observations:

- **The policy is a much more aggressive operator than it was under the old reward.** It removes 1608.5 mm^3/min, against 1205.2 before, and it cuts a part in 11.4 s against 20.3 s. The price is paid in the tool and in the part quality: the breakage rate is 6.0% and 4.0% of the finished surfaces are outside the roughness limit.
- **The roughing passes are rough:** 87% of the worst passes are above the safety margin ($0.85 \times 1.6 = 1.36$ μm) and 55% are above the hard limit of 1.6 μm. The finished surface, which is the quality the reward prices, is inside the limit in 96% of the scenarios.
- **Wear is never the binding constraint on a single part:** 0% wear violations, and the mean maximum wear of 0.1407 mm is under half the 0.3 mm limit. The constraint binds on the aggregate of a batch, which is why the batch experiment of Section 7.7 exists.
- **High variance:** the standard deviation of MRR is 34% of its mean, and the scenarios range from 2 to 15 mm of stock, so the policy is measured on a wide spread of operating points.

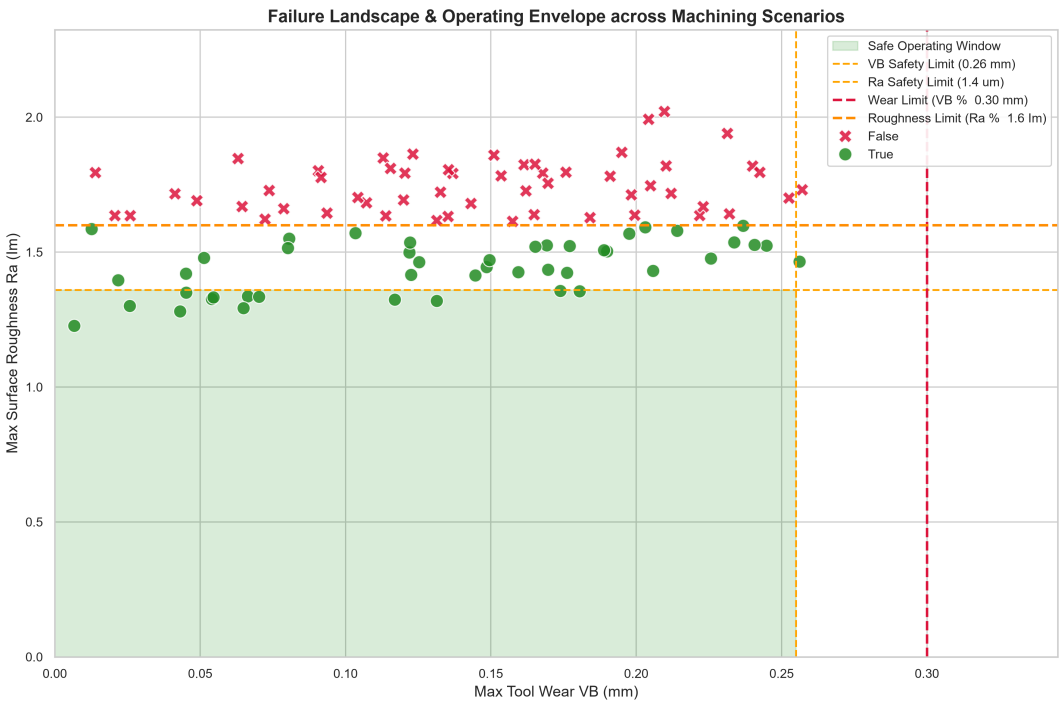


Figure 2: The operating envelope of the 100 Monte Carlo scenarios: the maximum flank wear of a scenario on the horizontal axis against its maximum surface roughness on the vertical axis. The shaded rectangle is the safe operating window, bounded by the wear safety limit $V_{B,\text{safe}}$ (0.255 mm, drawn at 0.26 mm) and the roughness safety limit $R_{a,\text{safe}}$ (1.36 μm, drawn at 1.4 μm). The dashed lines mark the hard limits $V_{B,\max} = 0.3$ mm and $R_{a,\max} = 1.6$ μm. A circle marks a scenario that stays inside both hard limits, a cross a scenario that ends above the roughness limit.

Figure 2 puts the two constraints on one plane and shows where the 100 scenarios land. No scenario reaches the wear limit: the rightmost point stops just inside the wear safety limit at 0.26 mm, well short of $V_{B,\max} = 0.3$ mm, which is the 0% wear violation rate of Table 4. The scenarios lie on a downward slope, the more the tool wears the smoother the surface, so the scenarios that push the roughness up are the ones with the least wear. The 55 crosses above $R_{a,\max} = 1.6$ μm are the worst-pass roughness violations of the table, and the circles are the 45 scenarios whose worst pass stays inside the limit, the ones between $R_{a,\text{safe}}$ and $R_{a,\max}$ exceeding the safety margin without breaking it. The finished surface is a separate quantity, and it is outside the limit in 4 of the 100 scenarios.

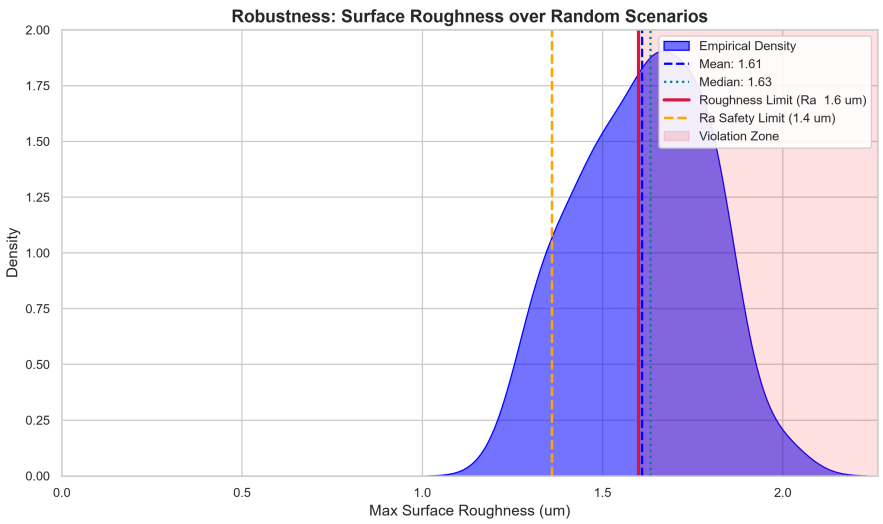


Figure 3: Density of the worst-pass surface roughness over the 100 Monte Carlo scenarios, with the roughness limit $R_{a,\max} = 1.6$ μm, the safety limit $R_{a,\text{safe}} = 1.36$ μm (drawn at 1.4 μm) and the violation zone beyond the limit.

Figure 3 is that distribution in one axis. The mean is 1.61 μm and the median 1.63 μm, and the density is concentrated on and just above the limit, which is the same fact the 55% worst-pass violation rate states. The distribution has a single mode, near the limit: under the old reward it carried a second mode near 0.9 μm from the scenarios cut conservatively, and that mode is gone, because the reward now pays for the throughput that the conservative scenarios give up. The finishing pass is a separate quantity, and it is outside the limit in 4% of the scenarios.

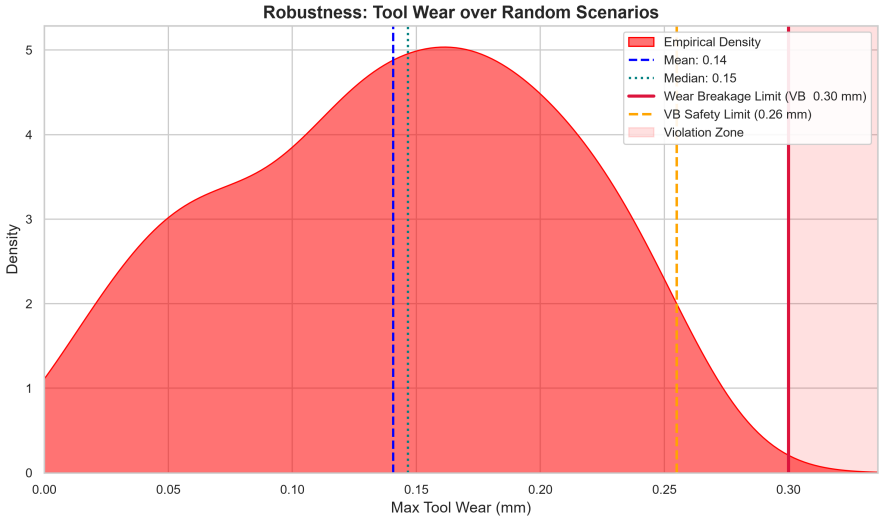


Figure 4: Density of the maximum flank wear over the 100 Monte Carlo scenarios, with the wear limit $V_{B,\max} = 0.3$ mm, the safety limit $V_{B,\text{safe}} = 0.255$ mm (drawn at 0.26 mm) and the violation zone beyond the limit.

Figure 4 is the same view of the wear. The mean sits at 0.14 mm and the median at 0.15 mm, and the density now reaches the safety limit at 0.26 mm instead of collapsing well before it, so the wear of a single part runs close to the limit without crossing it. The 6% breakage rate of Table 4 comes from the stochastic draw above 0.15 mm of wear, not from the wear limit

itself.

7.2 Energy Weight Ablation – Pareto Frontier

Table 5: Ablation study: mean ± std over 3 seeds, 50 scenarios each, every policy trained and evaluated with its weight pinned

w_2	MRR (mm^3/min)	Energy (kJ)	Cycle (s)	n (RPM)	f_r (mm/r)	Frontier
0.0	1397.1 ± 14.8	70.4 ± 0.1	14.8 ± 0.3	7998	0.299	dominated
0.2	1456.9 ± 14.6	70.0 ± 0.1	13.8 ± 0.2	7992	0.299	dominated
0.4	1524.9 ± 40.2	69.3 ± 0.2	13.0 ± 0.3	7896	0.296	dominated
0.5	1556.6 ± 9.7	69.1 ± 0.2	12.7 ± 0.1	7859	0.296	✓
0.6	1519.8 ± 36.4	68.3 ± 0.4	13.1 ± 0.4	7638	0.296	✓
0.8	1224.4 ± 11.3	63.0 ± 0.2	15.4 ± 0.1	6573	0.292	✓
1.0	918.5 ± 14.9	57.2 ± 0.2	20.1 ± 0.4	5382	0.292	✓

Proposition 2 (Frontier Shape)**.** *Across the Pareto frontier ($w_2 \in [0.5, 1.0]$): MRR falls 41% (1557 → 919 mm^3/min) while energy falls 17% (69.1 → 57.2 kJ). The trade-off is asymmetric: the energy of the part is bought with a lot of removal rate, and the spindle speed falls with the energy weight, which is the mechanism the model predicts.*

The dominated points ($w_2 = 0.0, 0.2, 0.4$) all spend more energy than a point on the frontier. The two low weights push the speed to the top of the envelope, which raises $k_c(v_c)$ without cutting the idle energy enough to compensate, and $w_2 = 0.4$ is dominated by $w_2 = 0.5$, which removes more material for less energy.

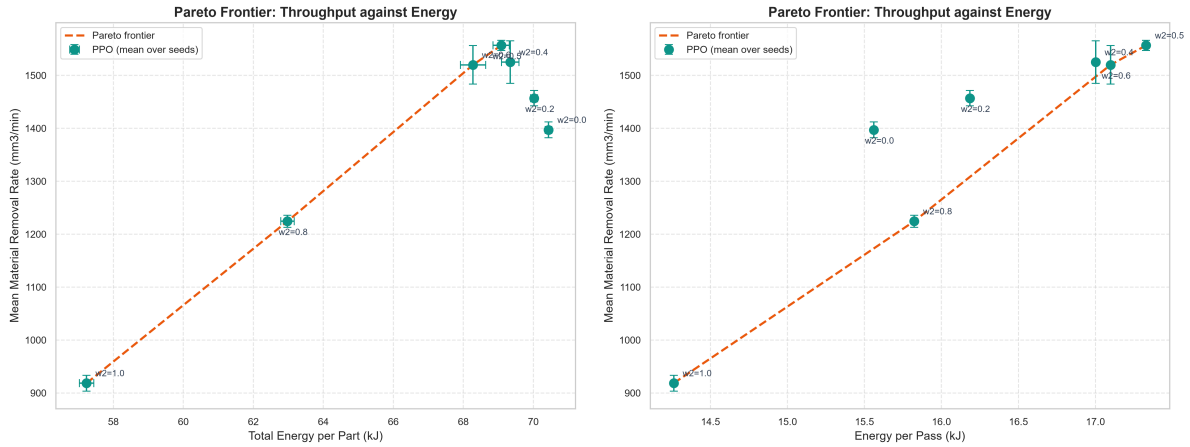


Figure 5: The ablation frontier: the mean removal rate of each of the seven pinned-weight policies against its energy, with the spread over the three seeds as error bars and the non-dominated points joined into the frontier. The left panel carries the energy of a part, the metric of Table 5, and the right panel the energy of a single pass, the quantity the reward charges.

Figure 5 draws the seven policies in the plane of energy against removal rate. The frontier is the chain of four non-dominated points from $w_2 = 0.5$ to $w_2 = 1.0$, and it is the same chain in both panels, so the energy of a pass and the energy of a part select the same operating points here. The three dominated points sit off it: $w_2 = 0.0$ and $w_2 = 0.2$ to the right of the chain, spending more energy for less material, and $w_2 = 0.4$ in the cluster at the top of the chain, just under the segment between $w_2 = 0.5$ and $w_2 = 0.6$.

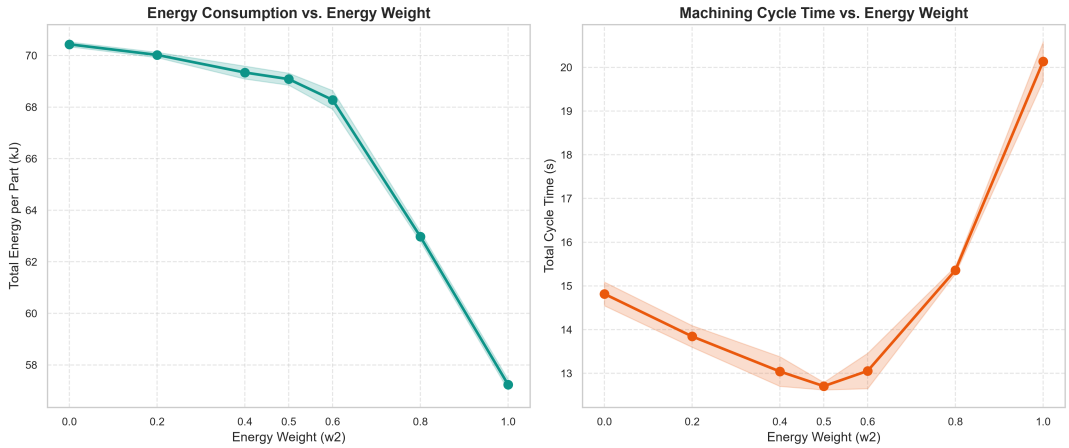


Figure 6: The energy weight sweep of the ablation policies: the energy per part (left) and the cycle time per part (right) against the energy weight, with the spread over the three seeds shaded.

Figure 6 shows the same sweep as two curves against the weight. The energy per part falls from 70.4 kJ at $w_2 = 0.0$ to 57.2 kJ at $w_2 = 1.0$, and the shaded band shows that the three seeds agree on the shape. The cycle time is not monotone: it falls from 14.8 s to its minimum of 12.7 s at $w_2 = 0.5$ and then rises to 20.1 s at $w_2 = 1.0$. Past $w_2 = 0.5$ the energy is bought with the removal rate and with the time, and the two heaviest weights are where the trade-off is steepest. That is the region in which the one conditioned policy loses to the per-weight policies of Section 7.4.

7.3 Runtime Weight Sweep of the Production Policy

The weight is a query, not a retrain: the same policy is asked for each point of the trade-off at runtime (`python -m experiments.evaluate --sweep-weight`) over the same 100 scenarios.

Table 6: Production policy asked for every weight at runtime (100 scenarios)

w_2	MRR (mm^3/min)	Energy (kJ)	Cycle (s)	n (RPM)	f_r (mm/r)	R_a violation rate
0.0	1475.9	68.4	13.1	8000	0.323	56.0%
0.2	1518.0	65.8	11.8	7997	0.323	56.0%
0.4	1608.5	66.4	11.4	7907	0.318	55.0%
0.5	1630.3	66.0	11.4	7798	0.317	54.0%
0.6	1540.8	65.5	12.4	7433	0.317	55.0%
0.8	1114.5	57.9	17.4	5841	0.316	56.0%
1.0	720.4	51.7	26.1	4111	0.315	56.0%

As the weight goes from 0.0 to 1.0 the removal rate falls from 1475.9 to 720.4 mm^3/min, the energy of the part falls from 68.4 to 51.7 kJ, the cycle time rises from 13.1 to 26.1 s and the spindle speed falls from 8000 to 4111 RPM. The operating point moves along the trade-off without a retrain. The response is not monotone in the removal rate: it peaks at 1630.3 mm^3/min at $w_2 = 0.5$, so the weight buys the energy of the first half of the range with throughput and only starts to pay for it after that. The roughness violation rate of the worst pass stays at 54% to 56% over the whole range, so the operating point moves without the constraint behaviour changing.

7.4 One Policy against the Per-Weight Policies

The production policy is compared against the 21 ablation policies, one per weight, each trained and evaluated with its weight pinned (`python -m experiments.evaluate --sweep-weight --compare-only`).

Table 7: One policy against 21 policies, one per weight. Removal rate in mm^3/min, energy per part in kJ.

w_2	MRR, one policy	MRR, per weight	MRR gap	Energy, one policy	Energy, per weight	Energy gap
0.0	1475.9	1397.1	+5.6%	68.4	70.4	−2.9%
0.2	1518.0	1456.9	+4.2%	65.8	70.0	−6.1%
0.4	1608.5	1524.9	+5.5%	66.4	69.3	−4.2%
0.5	1630.3	1556.6	+4.7%	66.0	69.1	−4.4%
0.6	1540.8	1519.8	+1.4%	65.5	68.3	−4.0%
0.8	1114.5	1224.4	−9.0%	57.9	63.0	−8.0%
1.0	720.4	918.5	−21.6%	51.7	57.2	−9.7%

The claim of the previous revision has reversed. Under the old reward the one policy was ahead of all 21 by 14% to 28% of the removal rate. Under the reward of this revision it is ahead by 1.4% to 5.6% over the weights from 0.0 to 0.6, and it is behind at the two heaviest weights, by 9.0% at $w_2 = 0.8$ and by 21.6% at $w_2 = 1.0$. Over the whole range of 0.0 to 0.6 it also spends 2.9% to 6.1% less energy than the per-weight policies, and at 0.8 and 1.0 it spends 8.0% and 9.7% less energy for the throughput it gives up.

The reason is the signal, not the training budget. The weight in the observation is a weaker signal than a dedicated policy where the trade-off is steepest: one conditional policy has to serve 0.0 and 1.0 with the same network, and the sweep of Table 6 shows how far apart those two requests are (8000 RPM against 4111 RPM, 13.1 s against 26.1 s). A policy trained for one weight is not asked to do both. The honest reading is therefore: one conditional policy covers the range and is ahead over most of it, but a policy trained for a single weight is better at the extreme weights, where the trade-off is steepest. The weight stays a runtime query rather than a retrain, but it is not free at the ends of the range.

7.5 Baseline Comparison

The five classical policies are evaluated on the same 50 episodes of the nominal machining condition at the same energy weight $w_2 = 0.4$, so the numbers are comparable. The reward column is the decisive one, because it carries the constraint penalties.

Table 8: Baseline comparison, 50 episodes of the nominal condition at $w_2 = 0.4$

Policy	**Reward**	MRR (**mm^3/min**)	**Energy (kJ)**	**Cycle (s)**	R_a **final (μm)**
Constant high	10.0	**1099.6**	17.8	**4.3**	1.56
NSGA-II static	5.1	732.9	18.8	6.4	1.56
Rule based	3.3	719.8	18.8	6.6	1.56
Receding-horizon	**253.5**	725.5	19.0	6.7	1.25
Constant safe	−1882.9	1.6	385.7	768.0	0.26

- **Three of the four static policies finish outside the roughness safety limit** ($R_{a,\text{safe}} = 0.85 \times 1.6 = 1.36$ μm), at 1.56 μm. A part that finishes out of spec is paid no completion bonus and is charged the finishing-pass penalty, so those three land near zero

(10.0, 5.1 and 3.3) however fast they cut. The fourth, the constant-safe policy, finishes inside the limit at 0.26 μm but needs 768 s and 385.7 kJ per part, which is what its -1882.9 measures.

- **The receding-horizon controller is the strongest baseline** at $+253.5$, and it is the only classical policy that finishes every part inside the limit, at 1.25 μm. It pays for that with throughput and time: 725.5 mm^3/min and 6.7 s against 1099.6 mm^3/min and 4.3 s for the constant-high policy.
- **The learned policy is measured against the same controller in the next two sections**, on 100 randomized scenarios at the same weight, where the controller scores 248.9 and the policy 233.6. On the 50 episodes of this table the figure annotates the mean reward of the learned policy at 248.4, level with the controller's 253.5, and the figure shows the learned policy removing more material per minute.

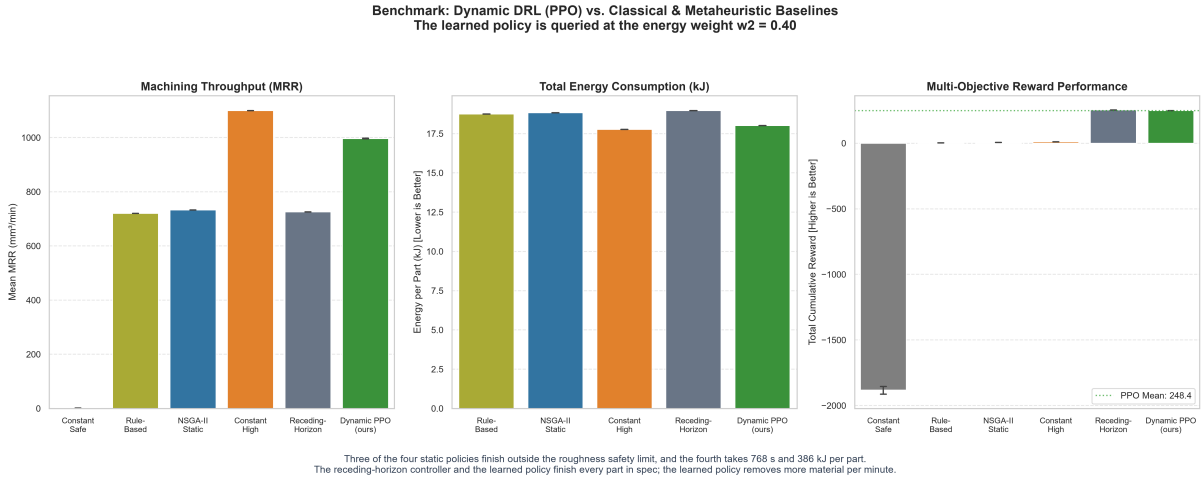


Figure 7: The six policies on the same 50 episodes of the nominal condition at $w_2 = 0.4$: the mean removal rate, the energy per part and the cumulative reward. The four static policies all take a reward near zero or deeply negative, the three fast ones because they finish outside the roughness safety limit and Constant-Safe because it needs 768 s and 385.7 kJ per part, so the comparison that matters is the receding-horizon controller against the learned policy, the only two with a positive reward.

Figure 7 shows the same six policies as three bar charts. The first two panels cannot be compared directly, because Constant-Safe sits at the edge of both scales: it removes 1.6 mm^3/min and needs 385.7 kJ, which is why it appears as a sliver in the throughput panel and is left out of the energy panel. The energy of the other five policies lies between 17.8 kJ and 19.0 kJ, so the energy does not separate them. The third panel does, because the reward carries the constraint penalties: the three fast static policies sit at 10.0, 5.1 and 3.3, the level of the finishing-pass penalty with no completion bonus to cancel it, Constant-Safe sits at -1883, and the receding-horizon controller and the learned policy sit at $+253$ and $+248$. The learned policy removes more material per minute than the controller, and the two are level on the reward.

7.6 Robustness Against the Classical Controller

This is the central result of the work. The two arms are measured on the same 100 randomized scenarios at the same energy weight $w_2 = 0.4$, and the three regimes change what the controller is given, not what the policy is given. The artifact is `results/evaluation/robustness_results.csv` and the script is `experiments/robustness_experiment.py`.

Table 9: Robustness experiment: the same two arms on the same 100 randomized scenarios at $w_2 = 0.4$, in three regimes. Std is the standard deviation of the episode reward over the scenarios.

Regime	**Arm**	**Mean reward**	**Std**	MRR (mm^3/min)	**Energy** (kJ)	**Cycle** (s)	**Tool breakage**	**Parts completed**
Controller has every advantage	Receding-horizon	**248.9**	79.5	1318.2	68.9	17.1	3%	96%
	PPO	233.6	79.7	**1608.5**	66.4	**11.4**	6%	94%
Material model is wrong	Receding-horizon	63.5	117.1	1306.9	71.0	23.3	2%	95%
	PPO	**226.9**	88.3	1604.2	66.3	11.5	6%	94%
Wear measurement is noisy	Receding-horizon	181.3	185.9	1300.4	74.2	31.1	4%	91%
	PPO trained with the noise	**213.6**	87.3	1554.2	66.9	12.0	6%	94%

Four things follow from the table.

1. Inside a perfect digital twin there is no reinforcement learning advantage. The controller computes the per-pass optimum of the same objective with the exact model, the exact state and the material in its hand, and it scores 248.9 against the policy's 233.6. The gap is inside the noise: the standard error of the mean is about 8 for both arms, so a difference of 15 has a t of about 1.4.

2. The learned policy is the better throughput operator. It removes 22% more material per minute (1608 against 1318), finishes a part in a third less time (11.4 against 17.1 s) and spends 3.7% less energy. It pays for that with twice the tool breakage, 6% against 3%, and with 4% of its finished surfaces above the roughness limit against the controller's 2%.

3. The advantage appears where a plant actually is. With a material the controller has no model for, its reward collapses by three quarters, from 248.9 to 63.5, because it does not slow down and its finished surfaces go out of specification. The learned policy holds at 226.9. The same holds for a noisy measurement: with 5% noise on the wear and on the remaining stock the controller's reward falls to 181.3 and its spread more than doubles, to 185.9 against the policy's 87.3. The policy trained with the same noise keeps 213.6 and cuts the part in 12.0 s against the controller's 31.1 s.

4. The policy does not need the material to be told. The third arm of the model-error regime, the policy that is told the true material, scores 215.1 with a spread of 94.2, against 226.9 and 88.3 for the policy that believes the nominal material. The material in the observation is a help, not a necessity.

The claim this work supports is therefore narrow and specific: the learned policy does not beat a model-based controller on the objective inside a perfect digital twin, because the controller computes the optimum of the same objective with strictly more information. It beats it in the two regimes where that information advantage disappears, a wrong model of the material and a noisy wear measurement. And the scale of the reward is a design decision that decides how the ideal case is scored.

7.7 Batch Against the Part

A tool is a resource that a batch of parts runs out of. The environment `MillingDT/env/milling_batch_env.py` (`MillingDTBatchEnv-v1`) cuts a batch of ten parts with one tool: the tool is replaced when it breaks or reaches its wear limit, the replacement costs 500 reward units against the 200 a finished part is worth, and the observation is the eleven features of the part plus the tool life left and the fraction of the batch still to cut. The artifact is `results/evaluation/batch_results.csv`, measured over 30 batches.

Table 10: Batch experiment: a batch of ten parts cut with one tool, 30 batches. The tool change costs 500 reward units.

Arm	Batch reward	Std	Reward per part	Tool changes	Parts cut	Batch time (s)	Batch energy (kJ)
PPO, batch aware	1772.3	358.3	169.0	**0.767**	10	**113.8**	**632.7**
PPO, per part	1694.6	352.3	171.7	0.833	10	110.8	655.7
Receding-horizon, per part	**1859.9**	328.8	173.7	0.833	10	207.5	692.0

The batch context is worth 8% fewer tool changes and 3.5% less energy at the same completion. The policy that sees the tool life and the batch that is left replaces the tool 0.767 times per batch against 0.833 for the same policy without that context, and it uses 632.7 kJ against 655.7 kJ, and both cut the ten parts.

The per-part controller gets the highest batch reward, 1859.9, and it buys it with the machine time. It needs 207.5 s of machine time for the batch against the batch-aware policy's 113.8 s, an almost twofold longer batch for a reward that is 4.9% higher. The reason is structural: the controller solves one part at a time, so it has no notion of a tool that the parts of a batch share, and it cannot price a tool change against the parts that follow it.

7.8 Controller Initialization Experiment

The baseline comparison left one question open: the controller re-solves the next pass with the exact state and the exact physics, so it is a strong teacher, and the learned policy is level with it on the reward. The experiment asks whether a policy that starts from the controller ends up better than the expert it started from.

The method is a behavior cloning step used as an initialization. The controller's behavior is recorded on 2000 randomized episodes, a supervised actor with the same shape as the PPO actor (two hidden layers of 256 with Tanh) is fitted on those trajectories by regression, and the fitted weights are copied into a fresh PPO policy. The reinforcement learning therefore starts from a feasible behavior instead of from noise. The policy is then trained with the same budget as the production policy (1,000,000 steps) and evaluated on the same 100 randomized scenarios at the same energy weight $w_2 = 0.4$. The clone lives in `MillingDT/agent/controller_init.py`, the experiment in `experiments/controller_experiment.py`, and the trained policy in `results/run_PPO_controller_init_seed42/`.

Table 11: Controller initialization experiment: three arms on the same 100 scenarios at $w_2 = 0.4$. Std is the standard deviation of the episode reward over the scenarios.

Arm	Mean reward	Std	MRR (mm^3/min)	Energy (kJ/part)	Cycle (s)	R_a viol. finished	Flank wear violations	Tool breakage	Parts completed
Receding-horizon controller	**248.9**	79.5	1318.2	68.9	17.1	2%	1%	3%	96%
PPO from scratch	233.6	79.7	**1608.5**	66.4	**11.4**	4%	0%	6%	94%
PPO initialized from the controller	119.8	178.7	1274.9	81.5	42.6	4%	9%	4%	87%

The initialization is now actively harmful, and the reason is instructive. The policy initialized from the controller scores 119.8 against the from-scratch policy's 233.6, a gap of 113.8, which is more than fourteen times the standard error of the mean over the 100 scenarios. The cloned actor starts the policy at the controller's operating point, which is the conservative one: a low feed and a low spindle speed, so that every pass of the part stays inside the roughness safety limit. With the wear constraint now binding, that conservatism is expensive. The policy then learns from a bad starting point, slows down further to stay away from the wear limit, and ends up cutting a part in 42.6 s against the controller's 17.1 s and the from-scratch policy's

11.4 s, while it spends 81.5 kJ per part against 66.4 kJ. It also exceeds the flank wear limit on 9% of the parts, which the other two arms never do.

The earlier revision found the initialization harmless. It made the policy more consistent and did not change the reward, and the report said so. That was the reward not counting the cost of the slow operating point: a policy that finished every part at any cost looked like a good policy. With the throughput counted, the initialization is what the measurement says it is.

So the method to keep from the controller initialization is the record and clone machinery, not the initialization itself. The recording, the regression and the weight copy are a clean way to put a feasible behaviour into a policy, and they stay in the code, because they are what makes the arms of Table 11 comparable. What the experiment now shows is that starting the reinforcement learning at the controller's operating point is the wrong way to use them.

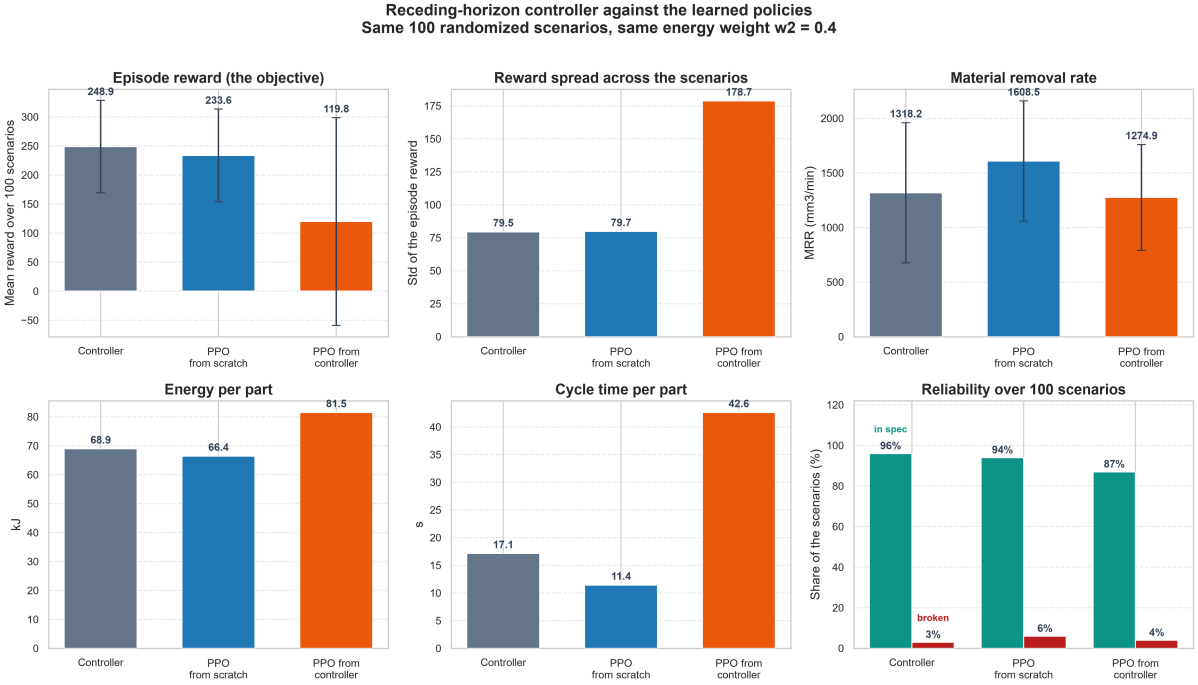


Figure 8: The three arms of the controller initialization experiment on the same 100 scenarios at $w_2 = 0.4$: the mean episode reward with its spread over the scenarios, the spread on its own, the removal rate, the energy per part, the cycle time and the reliability. The initialized arm sits **114 points** below the from-scratch arm on the reward, and its spread of **178.7** is more than twice the spread of the other two. It is the only arm that cuts a part in more than 17 s and the only one that exceeds the flank wear limit.

Figure 8 puts the three arms side by side over the six measures. The top-left panel is the one to read with care: the three bars are 248.9, 233.6 and 119.8, and while the error bars of the first two are far wider than the gap between them, the gap to the third is far wider than its error bar. The second panel shows the same thing in the spread: the reward standard deviation is 79.5 for the controller, 79.7 for the from-scratch policy and **178.7** for the initialized one, so the initialization does not buy consistency any more, it buys variance. The remaining panels show what it costs: the initialized arm removes 1274.9 mm^3/min against 1608.5, spends 81.5 kJ against 66.4 kJ and takes 42.6 s against 11.4 s. On the reliability panel it completes 87% of the parts against 94% and 96%, and it is the only arm with 9% flank wear violations.

7.9 Transfer Experiment

Target material: harder alloy ($k_1 \times 1.25$, $k_2 \times 1.30$) with faster-wearing tool ($C_{\text{wear}} \times 2.0$). The source policy was trained on the nominal material only and has never seen this material.

Table 12: Transfer experiment on the target material, 30 scenarios

Arm	Mean reward	R_a violation worst pass	R_a violation finished surface	Parts completed
Source policy, zero-shot	168.5	100.0%	6.7%	86.7%
Source policy, fine-tuned (50k steps)	101.9	26.7%	0.0%	86.7%
Trained on the target from scratch (50k steps)	101.9	26.7%	0.0%	86.7%
Production policy, zero-shot	**190.1**	100.0%	10.0%	90.0%

The production policy is the best arm without a single adaptation step. It was trained across a distribution of materials with the material context in its observation, so the harder alloy is inside its training distribution. Zero shot it scores the highest reward of the four arms (+190.1) and completes 90.0% of the parts against 86.7% for the other three. Its worst pass violates the roughness limit in every scenario, and its finished surface in 10.0% of them, which is worse on that count than the nominal-trained source policy, and it is still ahead on the reward.

Adaptation repairs the roughness violations and costs more than it gains. Fine tuning the source policy for 50,000 steps on the target, or training on the target from scratch for the same budget, both cut the worst-pass violations from every scenario to a quarter of them and take the finished surface back inside the limit, and both land at exactly the same reward, 101.9, against the source policy's 168.5 zero shot. The reason is the material: the target alloy wears the tool twice as fast, and a policy that slows down for that trades away more throughput than it gains in specification. The lesson is that the cost of a material change is not only the roughness, it is the throughput that the repair gives up.

8 Interactive Digital Twin Workstation

The Streamlit application (`app.py`) provides an industrial-grade interface:

1. **2D Cross-Sectional Stock Depletion:** Real-time visualization of cylindrical billet, pass-by-pass cutout, remaining allowance κ, depth per pass a_p, and tool engagement.
2. **Micro-Topography Generator:** Feed-mark cusp profile with an ISO 1302 centerline average R_a overlay and a 1.6 μm ceiling:

$$y_{\text{cusp}}(x) \approx \frac{x^2}{8r_\epsilon} + \text{stochastic perturbation}$$

3. **ISO 6983 G-Code Synthesizer:** Emits Fanuc/Siemens compliant G-code for the exact PPO parameter sequence.
4. **Multi-Controller Comparison:** PPO against NSGA-II, Rule-Based and Constant-High, with a telemetry overlay.
5. **Cached Policy Loading:** `@st.cache_resource` ensures model loads once per session.

The workstation loads the production policy (`results/run_PPO_seed42`) and exposes the energy weight as a runtime slider, so the whole trade-off is reachable from the browser without a retrain.

Launch: `streamlit run app.py`

9 Software Engineering Practices

9.1 Architecture

- `MillingDT/config/`: Calibrated constants and paths (YAML), the shared physics equations, the scenario distributions (single source of truth)
- `MillingDT/env/`: The Gymnasium environments, two registrations (`MillingDTEnv-v1` for the part, `MillingDTBatchEnv-v1` for the batch)
- `MillingDT/agent/`: The controller initialization, the behavior clone of the receding-horizon controller
- `MillingDT/baselines/`: The receding-horizon controller
- `MillingDT/sim/`: Simulation engine for Streamlit (stock depletion, micro-topography, G-code)
- `MillingDT/utils/`: Seeding, reproducibility
- `experiments/`: The entry points: training, evaluation, ablation, baselines, transfer, controller, robustness and batch experiments, pipeline orchestration
- `viz/`: Plotting utilities, training curves, MC distributions, baseline comparison, controller comparison
- `results/`: One directory per run, holding the policy, the log, the metrics and the figures of that run

9.2 Behavior Cloning as an Initialization

The receding-horizon controller is a competent expert, so it can supply the starting point of the learner instead of leaving the learner to start from noise. The module `MillingDT/agent/controller_init.py` records what the controller does, fits a supervised actor of the same shape as the PPO actor on those trajectories by regression, and copies the fitted weights into a fresh PPO policy. The initialization stays a separate concern: the clone knows nothing about the training loop, the copy is a weight assignment between two tensors of the same shape, and the rest of the run is an ordinary PPO training, selected with the `--init-from-controller` flag. The same separation is what makes the arm comparable: the three arms of Table 11 share the environment, the scenarios, the budget and the evaluation code, so the only difference between them is where the weights start. The measurement of Section 7.8 is that the difference matters: the initialization now costs 113.8 reward points, so the machinery is kept and the initialization is not.

9.3 Reproducibility

- Global seeding via `MillingDT/utils/seeding.py` (Python, NumPy, PyTorch, CUDA deterministic)
- `MillingDT/config/scenarios.py` shared by training and Monte Carlo – no drift

- `experiments/summarize_results.py` rebuilds `RESULTS.md` from committed CSVs
- The comparison tables are committed as CSVs under `results/evaluation/`: the baselines, the ablation summary, the weight sweep against the ablation, the transfer, the controller initialization, the robustness and the batch tables
- All random draws (scenarios, weights, seeds) are deterministic given the seed

9.4 Testing

45 tests in `tests/` covering:

- Environment: action clipping, truncation, soft norm, the 11-dimensional observation, physics constants, overrides, info dict, energy norm, Gym checker, randomization, pinned values, scenario scales, the context (weight and material), the belief of the material, the measurement noise, random weight, cutting energy speed dependence, part energy interior minimum, shared physics, shared scenario tables
- Reward: golden-value test (independent recomputation), finishing-pass penalty
- Batch environment: the 13-dimensional batch observation, the tool life and the batch left, every part of the batch, the tool change, the wear carried over, a broken tool, the wear limit, the batch reward, the seeded reproducibility, the pinned weight, Gym checker
- Integration: training→evaluation roundtrip, one registration, the context in the observation, one policy answering a pinned weight, baseline artifact
- Progress callback: heartbeat delivery, failure reporting, interval respect

9.5 Code Quality

- Centralized constants (`MillingDT/config/physics_constants.yaml`), one calibrated value per line with its unit in the file
- Centralized paths (`MillingDT/config/paths.yaml`) – no hardcoded strings
- Environment ID exported from `MillingDT/env/__init__.py`
- No unused imports (AST-verified)
- Conventional commits, clean git history

10 Discussion

10.1 What Worked

1. The speed-dependent cutting energy model ($k_c \propto v_c^m$) is essential. In an earlier revision of the reward, without it the energy/throughput trade-off vanished and the 21 ablation policies came out identical.
2. The scale of the reward matters as much as the weights. With the objective term worth about 1.2 points against a completion bonus of 200, the reward did not count the throughput and the policy learned to finish every part at any cost. At about fifty points per part the trade-off is inside the reward, and the operating point of the production policy changed completely: 1608 mm^3/min against 1205 before.

3. Domain randomization + shared scenario tables: the policy generalizes to unseen scenarios (94% part completion, 4% finished-surface violations).

4. Putting the energy weight in the observation reduces the ablation cost from 21 training runs to one. The one policy is ahead of the per-weight policies over the weights from 0.0 to 0.6 and behind at 0.8 and 1.0, so the weight stays a runtime query, but it is not free at the ends of the range.

5. Training across a distribution of materials removes the need for an adaptation step: on a harder alloy the production policy is the best of the four transfer arms zero shot, while a policy trained on the nominal material has to be repaired.

6. A controller that maximizes the expected episode reward is a far stronger baseline than one that maximizes the objective term per pass: on the same scenarios it went from 184.3 to 248.9, and building it exposed that the earlier baseline comparison was hollow.

7. The robustness experiment is the central result of the work: the learned policy is the better throughput operator, and its advantage appears exactly where the controller's information advantage disappears, a wrong model of the material and a noisy wear measurement.

8. The batch context is worth something: 8% fewer tool changes and 3.5% less energy at the same completion.

9. The test suite catches physics regressions (energy interior minimum, cutting energy speed dependence) and reward regressions (golden-value test).

10.2 Limitations

1. Inside a perfect digital twin the learned policy does not outperform the model-based controller: the controller scores 248.9 against the policy's 233.6 (Table 9), and the gap of 15 is inside the noise. The learned policy is the better throughput operator (22% more material per minute, a third less time, 3.7% less energy) and it pays for that with twice the tool breakage and with more finished surfaces outside the roughness limit. The work therefore does not claim a reinforcement learning advantage inside a perfect twin.

2. Initializing the policy from the controller is now harmful rather than harmless: 119.8 against the from-scratch policy's 233.6 (Table 11). The clone starts the policy at a conservative operating point, and with the wear constraint binding that conservatism is expensive.

3. The scale of the reward is a design decision, and the comparison is sensitive to it. The objective term is worth about fifty points per part against a completion bonus of 200; another scale would score the ideal case differently and could reorder the arms.

4. The wear model is anchored on the tool life of one paper and the exponents are the exponents of the wear rate, not the Taylor tool life exponents, so the tool life varies only by a factor of a few across the machine envelope and a faster cut barely shortens it.

5. The energy weight ablation uses 150,000 steps per weight against 1,000,000 for the production policy, so the ablation frontier is a lower bound on what a pinned policy can reach and may shift with more training.

6. The reward charges the energy of a pass, while the reported metric is the energy of a part. A part cut in many shallow passes draws less power per pass but spends more time on the idle machine, so the two can move in opposite directions. Both are reported.

7. Stochastic tool breakage (6% rate) introduces variance; the reward penalty could be shaped more carefully.

8. The micro-topography model is a simplified feed-mark cusp; real chatter and vibration are not modeled.

9. The energy model is the modelling choice of this work, not a result taken from the reference papers, and it is calibrated against a single reference condition.

10. The frontier is steep at the heavy weights: from $w_2 = 0.5$ to $w_2 = 1.0$ the removal rate falls 41% for 17% less energy, and the cycle time rises from 12.7 s to 20.1 s. A decision-maker who needs a large energy reduction pays for it in throughput and in time.

10.3 Future Work

1. **The sensitivity of the comparison to the reward scale:** the scale of the objective term is a design decision and it decides how the ideal case is scored. Running the robustness experiment at several objective scales would separate the part of the ranking that comes from the reward from the part that comes from the policy.

2. **Against a restricted controller:** the controller re-solves every pass with the exact state and the exact physics, so it is an oracle that a learned policy cannot match from its observation alone. The robustness experiment answers the model-error and the measurement-noise part of that question; what is left is the information gap inside a perfect twin.

3. **Sim-to-real:** Calibrate physics coefficients on real machine data; add sensor noise and delay to the environment.

4. **Multi-objective RL:** Replace scalarization with Pareto-conditioned networks or hypervolume maximization.

5. **Runtime energy budget:** Drive the production policy with an optimizer that sets w_2 from an energy budget. The weight is a query, not a retrain.

6. **Batch-level planning:** let the policy plan a whole batch instead of a part, with the tool life and the tool change in the objective, so that the context of Section 7.7 is optimized rather than merely read.

7. **Full 3D Digital Twin:** Extend the 2D cross-section to 3D toolpath visualization with collision checking.

11 Conclusion

This project demonstrates a complete pipeline for DRL-based CNC milling optimization:

1. A physics-grounded Digital Twin with speed-dependent cutting energy, tool wear, surface roughness, and stochastic breakage. The wear coefficient is anchored on a tool life rather than on a wear equation, because none of the three reference papers carries one.

2. Two Gymnasium environments, one for a part and one for a batch of ten parts cut with one tool, with domain randomization, an always 11-dimensional conditioned observation and an energy-aware reward whose objective terms are scaled to a quarter of the value of a part.

3. PPO policies trained with proper experimental rigor: Monte Carlo evaluation with confidence intervals, an ablation study mapping the empirical Pareto frontier, and one production policy covering the frontier in a single run.

4. Strong classical baselines including a receding-horizon controller that re-optimizes at every pass and maximizes the expected episode reward over the same physics.

5. A robustness experiment, the central result of the work: the learned policy and the controller on the same 100 scenarios in three regimes. Inside a perfect digital twin the controller is ahead on the reward (248.9 against 233.6) and the gap is inside the noise. With a wrong model of the material the controller's reward collapses to 63.5 while the policy holds at 226.9; with a noisy wear measurement the controller falls to 181.3 and the policy keeps 213.6. The learned policy is the better throughput operator in every regime: 22% more material per minute, a third less time and 3.7% less energy, at twice the tool breakage.

6. A batch experiment: a batch of ten parts cut with one tool, where the tool change costs 500 reward units. The batch context is worth 8% fewer tool changes and 3.5% less energy at the same completion, and the per-part controller needs 207.5 s of machine time for the batch against the batch-aware policy's 113.8 s.

7. A controller initialization experiment, which records the receding-horizon controller on 2000 episodes, clones its actor into a fresh PPO policy and trains it with the same 1,000,000-step budget. The result reversed: the initialization is now harmful (119.8 against the from-scratch policy's 233.6), because the clone starts the policy at the controller's conservative operating point and the wear constraint makes that conservatism expensive. The record and clone machinery stays, the initialization does not.

8. A transfer experiment on a harder alloy, which finds that a policy trained across a distribution of materials needs no adaptation there, while a policy trained on the nominal material has to be repaired, and that the repair costs more throughput than it gains in specification.

9. An interactive workstation for visualization, G-code generation, and multi-controller comparison.

10. Full reproducibility: every result table is generated from committed artifacts by a single script.

The key scientific finding on the physics side is that **the speed-dependent specific cutting energy creates a genuine energy/throughput trade-off**, and a DRL agent can learn to navigate it pass-by-pass while respecting surface quality and tool life constraints. Across the measured frontier, from $w_2 = 0.5$ to $w_2 = 1.0$, the removal rate falls 41% while the energy of the part falls 17%, and the spindle speed falls with the energy weight, which is the mechanism the model predicts. One policy with the weight in its observation makes the whole trade-off accessible at runtime: it is ahead of the 21 policies trained per weight over the weights from 0.0 to 0.6, by 1.4% to 5.6% of the removal rate, and behind at 0.8 and 1.0, by 9.0% and 21.6%. The weight in the observation is a weaker signal than a dedicated policy where the trade-off is steepest.

The result on the classical side is narrower and more interesting than an advantage. **Inside a perfect digital twin the learned policy does not beat the model-based controller on the objective**, because the controller computes the optimum of the same objective with strictly more information: it has the exact model, the exact state and the material in its hand. What the learned policy does have is the throughput: it removes 22% more material per minute,

finishes a part in a third less time and spends 3.7% less energy, and it pays for that with twice the tool breakage and with more finished surfaces outside the roughness limit. Its advantage appears where the information advantage of the controller disappears, a wrong model of the material and a noisy wear measurement, and in the batch, where the tool life is a resource shared by the parts. The scale of the reward decides how the ideal case is scored, so the comparison is a statement about a design decision as well as about the two operators.

All code, data, and results are available in the repository at `G:/My Drive/IITM/AI in Mfg`.

References

References

[1] X. Mo, M. Xia, S. Mei, X. Hu, "Deep reinforcement learning-based optimization for machining process: Cutting parameters dynamic optimization considering tool condition degradation," *Journal of Manufacturing Processes* 156 (2025) 1006–1031.

[2] P. Wang, Y. Cui, H. Tao, X. Xu, S. Yang, "Machining parameter optimization for a batch milling system using multi-task deep reinforcement learning," *Journal of Manufacturing Systems* 78 (2025) 124–152, doi: 10.1016/j.jmsy.2024.11.013.

[3] P. Wang, Y. You, W. Liu, D. Ding, F. Xu, "Knowledge-guided deep reinforcement learning framework for feature machining step sequencing," *International Journal of Advanced Manufacturing Technology* 140 (2025) 5617–5636, doi: 10.1007/s00170-025-16606-4.

A Reproducibility Commands

All commands run from the project root `G:/My Drive/IITM/AI in Mfg`:

Listing 1: Key reproducibility commands

```
# Run all tests
pytest tests/ -v

# Train the production policy (1M steps, scenario and weight drawn per
    episode)
python -m experiments.train_rl --steps 1000000 --seed 42 --randomize --
    randomize-weight

# Train the production policy with the energy weight pinned
python -m experiments.train_rl --steps 150000 --seed 42 --randomize --
    w2 0.6

# Train the policy that faces a noisy wear measurement
python -m experiments.train_rl --steps 1000000 --seed 42 --randomize --
    randomize-weight --obs-noise 0.05 --run-name PPO_noisystate_seed42

# Train the batch-aware policy (one tool cuts a batch of ten parts)
python -m experiments.train_rl --env-id MillingDTBatchEnv-v1 --steps
    1000000 --seed 42 --randomize --run-name PPO_batch_seed42

# Monte Carlo sweep of the production policy (100 scenarios)
python -m experiments.evaluate --scenarios 100

# Runtime weight sweep of the same policy (100 scenarios)
python -m experiments.evaluate --sweep-weight --scenarios 100
```

```
# Compare the saved weight sweep against the per-weight policies
python -m experiments.evaluate --compare-only

# Ablation study (7 weights x 3 seeds, 150k steps each, weight pinned)
python -m experiments.train_rl_ablation --steps 150000 --scenarios 50
    --slots 7

# Classical baselines
python -m experiments.run_baselines

# Transfer experiment (300k source steps, 50k adaptation steps)
python -m experiments.transfer_experiment 300000 50000

# Controller initialization experiment (1M steps, 2000 controller
    episodes)
python -m experiments.controller_experiment 1000000 2000

# Robustness experiment (three regimes against the classical controller
    )
python -m experiments.robustness_experiment

# Batch experiment (the batch against the part)
python -m experiments.batch_experiment

# Regenerate all plots
python -m viz.plot_results
python -m viz.plot_eval
python -m viz.plot_baselines_comparison
python -m viz.plot_controller_comparison

# Regenerate RESULTS.md from artifacts
python -m experiments.summarize_results

# Launch interactive workstation
streamlit run app.py

# Full pipeline (sequential, eleven stages)
python -m experiments.run_pipeline --steps 1000000 --ablation_steps
    150000
```

B Project Structure

The project holds two Gymnasium registrations, `MillingDTEnv-v1` for the part and `MillingDTBatchEnv-v1` for the batch, one training script and one evaluation script. The registrations are exported as the constants `MillingDT.env.ENV_ID` and `MillingDT.env.BATCH_ENV_ID`. There is no separate conditioned environment, no separate conditioned training script and no `results/conditioned/` directory. The reusable code lives in the `MillingDT/` package, the entry points in `experiments/`, and every training configuration keeps its own directory under `results/`.

Listing 2: Project directory tree

```
G:\My Drive\IITM\AI in Mfg
|-- MillingDT/                        # The library package
|   |-- __init__.py                    # Exports ENV_ID, ALL_CONSTANTS
|   |-- config/
```

```
|   |   |-- __init__.py              # Loads the two YAML files
|   |   |-- physics_constants.yaml   # The calibrated constants and the
    reward scales
|   |   |-- paths.yaml               # Where the runs are kept
|   |   |-- physics.py               # Shared physics equations
|   |   `-- scenarios.py             # Randomized scenario sampling
|   |-- env/
|   |   |-- milling_env.py           # Gymnasium environment (11-D
   observation)
|   |   |-- milling_batch_env.py     # Batch environment (13-D
   observation)
|   |   `-- __init__.py              # Exports ENV_ID, BATCH_ENV_ID
|   |-- agent/
|   |   |-- controller_init.py       # Behavior clone of the controller
|   |   `-- __init__.py
|   |-- baselines/
|   |   |-- receding_horizon.py      # Receding-horizon controller
|   |   `-- __init__.py
|   |-- sim/
|   |   |-- machining_sim.py         # Stock depletion, topography, G-
   code
|   |   `-- __init__.py
|   `-- utils/
|       |-- seeding.py               # Global deterministic seeding
|       `-- __init__.py
|-- experiments/                     # The entry points
|   |-- train_rl.py                  # The one training script
|   |-- train_rl_ablation.py         # Energy weight ablation, pinned
|   |-- evaluate.py                  # Monte Carlo sweep, weight sweep
|   |-- run_baselines.py             # 5 classical baselines
|   |-- transfer_experiment.py       # Cross-material transfer
|   |-- controller_experiment.py     # Controller initialization, 3
   arms
|   |-- robustness_experiment.py     # Three regimes against the
   controller
|   |-- batch_experiment.py          # The batch against the part
|   |-- run_experiments.py           # Parallel training supervisor
|   |-- run_pipeline.py              # Full pipeline orchestrator
|   |-- summarize_results.py         # Regenerates RESULTS.md
|   `-- __init__.py
|-- viz/
|   |-- plotting_utils.py            # Shared plotting functions
|   |-- plot_eval.py                 # Monte Carlo distributions
|   |-- plot_results.py              # Training curves
|   |-- plot_baselines_comparison.py # Baseline comparison charts
|   |-- plot_controller_comparison.py # Controller against the learned
   policies
|   `-- __init__.py
|-- tests/
|   |-- test_env.py                  # Environment and reward tests
|   |-- test_batch_env.py            # Batch environment tests
|   |-- test_training.py             # Training tests
|   |-- test_progress_callback.py    # Callback tests
|   `-- test_integration.py          # Integration tests
|-- results/                         # One directory per configuration
|   |-- run_<config>/                # models/, log/, metrics/, plots/
|   |-- evaluation/                  # The tables that compare runs
|   |-- plots/                       # The cross-run figures
```

```
|   `-- RESULTS.md
|-- app.py                              # Streamlit workstation
|-- AUDIT_REPORT.md                     # Code audit with dispositions
|-- PROJECT.md                          # Consolidated documentation
|-- README.md                           # Project overview
`-- report/
    `-- report.tex                      # This file
```

C Hyperparameter Sensitivity

The PPO hyperparameters in Table 3 are the Stable-Baselines3 defaults, with three exceptions: the network is widened to 2×256 units, the entropy coefficient is raised to 0.02 to keep the policy from collapsing to deterministic actions early, and the training runs on the CPU with one thread per process so that several policies can train in parallel on one machine. Measured on this machine, PPO on this MLP runs at 812 steps/s on the CPU against 318 steps/s on the GPU, so the GPU is not used.

The entropy coefficient matters most: with too little of it the policy settles into a deterministic action early and stops exploring the envelope.

D Energy Model Derivation

The specific cutting energy $k_c(v_c)$ models the empirical observation that the energy required to remove a unit volume of material increases with cutting speed due to:

1. Higher shear zone temperatures $\rightarrow$ increased flow stress
2. Increased strain rates $\rightarrow$ strain-rate hardening
3. Tool-chip interface friction changes

The power-law form $k_c(v_c) = k_{c0}(v_c/v_{\text{ref}})^m$ with $m \approx 0.6$ is a standard empirical form for the rise of the specific cutting energy with cutting speed, and it is the modelling choice of this work rather than a result taken from either reference paper: the reference papers optimize quality and efficiency, and treat energy only as related work. The reference condition v_{ref} is the cutting speed at the calibrated nominal parameters ($n = 5000$ RPM, $D = 50$ mm $\rightarrow v_{\text{ref}} \approx 785$ m/min). At this speed, $k_c = k_{c0} = 2.5$ W·min/mm^3.

Total power: $P_{\text{total}} = P_{\text{idle}} + k_c(v_c) \cdot \text{MRR}$. Energy per pass: $E_{\text{pass}} = P_{\text{total}} \cdot t_{\text{cut}}$. Substituting $t_{\text{cut}} = L_{\text{cut}}/(nf_r)$ and $\text{MRR} = v_c a_p f_r$:

$$E_{\text{pass}} = P_{\text{idle}} \frac{L_{\text{cut}}}{nf_r} + k_{c0} \left(\frac{v_c}{v_{\text{ref}}} \right)^m v_c a_p f_r \frac{L_{\text{cut}}}{nf_r} \tag{14}$$

$$= \underbrace{P_{\text{idle}} \frac{L_{\text{cut}}}{nf_r}}_{\text{Idle energy}} + \underbrace{k_{c0} \frac{L_{\text{cut}} a_p}{nf_r} \left(\frac{\pi D n}{1000 v_{\text{ref}}} \right)^m \frac{\pi D n}{1000}}_{\text{Cutting energy}} \tag{15}$$

The idle term $\propto 1/(nf_r)$ decreases with speed; the cutting term $\propto n^m$ increases with speed. Their sum has an interior minimum, which is the operating point selected by w_2.